

A Review of SLAM Techniques and Security in Autonomous Driving

Ashutosh Singandhupe, Hung Manh La

Abstract—Simultaneous localization and mapping (SLAM) is a widely researched topic in the field of robotics, augmented/virtual reality and more dominantly in self-driving cars. SLAM is a technique of building a map of the environment and estimating the state of the robot in the map in which it is moving, simultaneously. SLAM has been there for more than 30 years and has contributed significantly in the industry targeting from small scale driven applications to large scale, which resulted in the advent of this decade's self driving cars. This paper attempts to give an understanding and progress of SLAM in autonomous driving industry as well as briefly describes the SLAM techniques that have contributed significantly to the industry, which were especially evaluated on KITTI dataset. We have also attempted to compare various techniques that were presented and made a rough estimate on why the state of the art approach can be revised and refurbished to suit the complex understanding of the environment for effective localization. In the end we have briefly described the security threats related to autonomous driving industry and why this is alarming.

I. INTRODUCTION

The ease with which most animals and we human beings navigate in the environment, perhaps, can be replicated in robots as well. This complex process of navigation, no wonder how well we do, can not be easily represented mathematically. The only way that dumb robots can be made to navigate in an environment is to represent the environment in some simpler forms, which can be algorithmically justified. Simultaneous Localization and Mapping (SLAM) is an algorithmic process of a robot/sensor system, which involves perceiving the environment using sensors and estimating the position of itself in the environment simultaneously [7]. For a robot, the environment can be represented as a culmination of different geometrical structures (landmarks, obstacles etc.) also called as map, and the pose can be represented as position and orientation of the robot, and it is generally termed as robot state. The map assists the human operator in visualizing an unknown environment and setting up the path of the robot for navigation. Another significant advantage the map provides is that it helps in minimizing the error while estimating the robot state (pose) during navigation. Keeping track of visited landmarks the robot can detect a loop closure in case it has revisited a location, thereby minimizing localization error, which is quite a similar to as how we human beings navigate. Automatic navigation is also significant part of robot navigation research, which has resulted in numerous algorithms such as Rapidly exploring Random Trees (RRT), extended RRT

(RRT*), Rapidly-exploring Random Graph (RRG), Probabilistic Roadmap (PRM), etc., [9]. These algorithms have directed many researchers to explore and improve robot navigation in complex environments. However, the current state of the art demands for a more improvement since it is yet to be fully solved for real time dynamic environments.

SLAM is a heavy component in autonomous driving car systems. This paper attempts to explore the various techniques tested on autonomous driving cars with reference to KITTI dataset [1] as our benchmark. The drive for SLAM research was ignited with the inception of robot navigation in Global Positioning Systems (GPS) denied environments. Although GPS improves localization, numerous SLAM techniques are targeted for localization with no GPS in the system. Initially, probabilistic estimation techniques were introduced like Kalman Filters (KF), which were later extended to Extended Kalman Filters (EKF), and Unscented Kalman Filters (UKF) for non-linear systems [7]. Particle filters like Rao-Blackwellized and Monte Carlo filters have also contributed significantly to the SLAM research [7]. Another approach that has grabbed attention is the graph based SLAM where the robot pose is represented as a node/vertex in a graph, and the edges represent the errors in measurements from various sensors. Subsequently the process involves generating a pose graph and minimizing the error using mathematical techniques like Gauss-Newton/LevenbergMarquardt [18]. SLAM techniques like Oriented fast and Rotated Briefs-SLAM (ORB2-SLAM) [24], [25] are the graph-based localization. Another interesting approach grabbed attention since the advent of deep learning with focus on Convolutional Neural Networks (CNN). Quite interesting results were observed especially with the work on CNN-SLAM [37]. It was shown through experiments that robot pose or localization could be achieved from a pair of images acquired by a moving robot through deep learning or CNN. Even though the CNN-SLAM approach is promising, this approach has invited few challenges that needs to be addressed. Deep learning requires high-end Graphic Processing Unit (GPU) systems, which is still a challenge for robotic embedded systems. Moreover SLAM systems are seen to be directed on continuous open-world scenes where the environment keeps changing. These changes needs to be learned on a continuous basis for a deep learning system. To our best knowledge, deep learning has not significantly evolved in the current state of SLAM to learn the dynamic changes in the environment robustly. From the various techniques introduced in SLAM, one can observe that SLAM has an inclination to combine various fields like signal processing, deep learning (CNN-SLAM) and more significantly computer vision.

The whole approach of SLAM is based on a robot localizing itself given the sensor measurements. However the traditional SLAM algorithms do not extend to perform task of a robot driving itself to collect more information about the environment. In a more general sense, traditional SLAM algorithms only estimates it's localization, when it is navigated

Ashutosh Singandhupe and Dr. Hung La are with the Advanced Robotics and Automation (ARA) Laboratory, Department of Computer Science and Engineering, University of Nevada, Reno, NV 89557, USA. Corresponding author: Hung La, email: hla@unr.edu

This material is based upon work supported by the National Aeronautics and Space Administration (NASA) Grant No. NNX15AI02H issued through the NVSGC-RI program under sub-award No. 19-21, the RID program under sub-award No. 19-29, and the NVSGC-CD program under sub-award No. 18-54.

or assisted by external source or through an external command. There is no ‘conscious’ effort from the robot to navigate itself and collect the data from the environment. Active SLAM, exploration and localization done at the same time, is an approach that attempts to solve this problem. Active SLAM basically tries to solve it in 3 steps. The first step specifies the robot trying to find possible actions (e.g., turn right, turn left, forward, backward, path selection, etc.) that it could take given the map space, however it exposes the challenge of increased computational complexity. The second step says that, even if an action is confirmed to be taken, the logic behind performing that action needs to be justified with respect to the goal of the task, as well as the complexity of the future action that could be taken to achieve that goal. The final step indicates that even if the action is performed it is quite difficult to arrive to a conclusion whether the exploration task has been completed or not. Based on our knowledge, active SLAM still requires mathematical proofs at various aspects [7].

Semantic SLAM is another active area of research for solving the SLAM problem. Based on this approach, the information of the environment is stored or perceived as semantics, where each part is seen as distinguished geometrical object. In more simpler terms, semantic SLAM tries to build the map of the environment using objects as references. In traditional SLAM methods, the whole approach of map building was done using the idea of aligning points and planes. However, the semantic SLAM approach uses the idea of aligning point clouds using objects, which are referred or understood as semantics. This has pushed SLAM community to create more techniques, which are tasks based rather than path planning based. It will also create a room for effective robot-human interaction. The way it differs from previous SLAM techniques is that previous approaches were based on already seen environments (regardless of identification of different objects) while semantic SLAM works on identifying a place based on objects described as semantic labels. However, it presents a challenge, which involves fusing semantic information from various sources at different times. The problem of consistently fusing it, still persists [7], [8].

Despite the tremendous progress made in SLAM in the past 30 years, one question still bothers the robotics community, Is SLAM solved? [7]. Our understanding is that SLAM is an estimation problem. Given the complexities of the environment and the uncertainties in the sensor measurements, it is yet to arrive a complete solution. As of our knowledge, SLAM is still unsolved. A good solution significantly relies on the environment, the robot, the uncertainties in the sensor measurements and the level of performance that we intended to achieve.

This paper primarily focuses on reviewing various SLAM techniques that were tried and tested on autonomous driving cars based on the KITTI dataset [1]. We are risking an attempt to classify various SLAM techniques, which are based on sensors used for localization and the ability of the SLAM algorithms to detect a loop closure. A loop closure is a technique of detecting a visited landmark or a scene in an environment. As of our knowledge, very few of the state-of-the-art algorithms we have encountered has solved the loop closure problem with respect to this autonomous driving car dataset.

The remainder of the paper is organized as follows. Section

II introduces techniques, which is further divided into Light Detection and Ranging (Lidar) based techniques (including Lidar and monocular camera) and stereo-based SLAM techniques. Later on, in section III we also intend to explore the security aspects in relation to autonomous driving and explore briefly the threats associated with it. Finally, conclusions are discussed in Section IV.

II. SLAM IN AUTONOMOUS DRIVING

Since autonomous cars require localization to navigate in the environment, it is quite obvious to explore the SLAM techniques in relation to autonomous cars. Most autonomous cars use Lidars and stereo cameras to perceive the environment in which it is navigating. However, most techniques that we have encountered so far, either have attempted to solve the localization problem on Lidar and Monocular camera or Stereo camera.

A. Lidar Based Odometry

There are numerous algorithms written for estimating odometry using Lidar. Implicit Moving Least Squares SLAM (IMLS-SLAM) [12] is quite popular, which uses scan-to-model matching framework. Initially, it uses an algorithm to sample the 3D scans and uses IMLS for surface reconstruction, which is claimed to have an improved matching quality. One key factor to note in this work is that it uses only 3D Lidar sensors for odometry estimation. The work claims to perform better results than the state of the art algorithm for odometry estimation, Lidar Odometry and Mapping (LOAM) [43], [44]. However, the KITTI website [15] shows that LOAM outperforms every other algorithm that has been tested on all the odometry datasets. This contradiction needs to be evaluated further.

Another work by [17], also called as Lidar-Monocular Visual Odometry (LIMO), have proposed a method, which fuses data from both Lidar and monocular camera. It first calculates the camera features and estimates the depth using the Lidar data corresponding to those features. Fusing the data together, it estimates the motion using a technique called bundle adjustment. The system for this algorithm can be better explained in steps/blocks. The first block relates to the camera, where it extracts the features. It includes feature tracking step and feature association step. Feature tracking is done using the Viso2 library. The feature association step is mostly related to extracting the depth of the camera features using the highly precise Lidar data. To perform these, the Lidar point cloud is transformed into camera frame and projected into the image plane. Then, for each features the following steps are performed. (1) From a projected set of Lidar points, the algorithm choose a set of points that are around a given feature. They use a rectangle to define the neighbourhood around that point. (2) Then it performs a plane estimation using histograms of depths with fixed bandwidth. This helps in estimating the depth around corner features as well. (3) Then the algorithm estimates the plane that fits the feature using the triangulation method. However depth estimation for the features on the road includes another preemptive processing, which includes RANSAC for plane fitting. After performing the above steps, the next step includes frame to frame odometry estimation using perspective-n-point problem. Besides the procedures described above, there are certain steps